

Software Requirements Specification (SRS)

Project: Hardware Store Management System

Version: 1.0

Date: February 24, 2026

Author: Group 15

1. Introduction

1.1 Purpose

This document describes the requirements for the **Hardware Store Management System (HSMS)**. The system is a web-based full-stack application developed using:

- Frontend: React.js
- Backend: ASP.NET Web API
- Database: MySQL
- Deployment: Azure App Service

The purpose of this system is to automate and manage hardware store operations including inventory management, sales processing, supplier management, reporting, and user management.

This SRS is intended for:

- The development team (project group members)
- The module instructors / project evaluators
- Project supervisor
- Stakeholders
- QA engineers

1.2 Scope

The Hardware Store Management System will:

- Manage products (tools, materials, equipment)
- Track stock levels
- Manage suppliers
- Process sales and generate invoices
- Generate reports (sales, inventory, profit)
- Support multiple user roles (Admin, Cashier, Manager)

The system will improve:

- Accuracy of inventory tracking
- Sales efficiency
- Reporting automation
- Business decision-making

1.3 Definitions

Term	Description
HSMS	Hardware Store Management System
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous (Deployment/Delivery)
SKU	Stock Keeping Unit

2. Overall Description

2.1 Product Perspective

HSMS is a web-based system consisting of:

- React frontend (Client-side UI)
- ASP.NET Web API backend
- MySQL relational database
- Hosted on Azure

High-Level Architecture

User → React Frontend → ASP.NET API → MySQL Database

2.2 Product Functions

The system shall:

1. Authenticate users (Login/Logout)
2. Manage products (Add, update, delete, search)
3. Manage inventory levels
4. Process sales transactions
5. Generate invoices
6. Manage suppliers
7. Generate dynamic reports
8. Maintain logs
9. Support role-based access control
- 10.Support CI/CD deployment

2.3 User Classes

User Type	Description
Admin	Full system access
Manager	Manage reports, inventory, suppliers
Cashier	Process sales only

2.4 Operating Environment

- Web browser (Chrome, Firefox, Edge)
- Windows/Linux server
- Azure App Service or Docker container
- MySQL 8+

2.5 Design Constraints

- Must use React for frontend
- Must use ASP.NET + ADO.NET (direct SQL)
- Must use MySQL
- Must use GitHub
- Must implement CI/CD (GitHub Actions)
- Must implement unit, integration & load testing

3. System Features & Functional Requirements

3.1 User Authentication

Description

Users must log in to access the system.

Functional Requirements

FR1: The system shall allow users to log in using username and password.

FR2: The system shall validate credentials from the database.

FR3: The system shall implement role-based access control.

FR4: The system shall allow users to log out securely.

3.2 Product Management

Description

Admin/Manager can manage hardware products.

Functional Requirements

FR5: The system shall allow adding new products.

FR6: The system shall allow updating product details.

FR7: The system shall allow deleting products.

FR8: The system shall allow searching products.

FR9: The system shall store product name, SKU, price, quantity, category.

3.3 Inventory Management

FR10: The system shall update stock after each sale.

FR11: The system shall alert when stock is below threshold.

FR12: The system shall allow manual stock updates.

3.4 Sales Management

FR13: The system shall allow cashier to create sales transactions.

FR14: The system shall generate invoice automatically.

FR15: The system shall calculate total and taxes.

FR16: The system shall store transaction history.

3.5 Supplier Management

FR17: The system shall allow adding suppliers.

FR18: The system shall allow updating supplier details.

FR19: The system shall delete suppliers.

FR20: The system shall associate suppliers with products.

3.6 Reporting

FR21: The system shall generate daily sales reports.

FR22: The system shall generate monthly sales reports.

FR23: The system shall generate low stock reports.

FR24: The system shall allow exporting reports (PDF/CSV).

4. Non-Functional Requirements

4.1 Performance Requirements

NFR1: The system shall respond within 3 seconds for normal operations.

NFR2: The system shall handle at least 100 concurrent users.

NFR3: Reports should generate within 5 seconds.

4.2 Security Requirements

NFR4: Passwords shall be encrypted.

NFR5: The system shall implement JWT authentication.

NFR6: SQL injection protection must be implemented.

NFR7: HTTPS must be used in deployment.

4.3 Reliability

NFR8: System uptime must be 99%.

NFR9: Database backups must be automated.

4.4 Usability

NFR10: The UI shall be responsive.

NFR11: The system shall be usable without technical training.

4.5 Maintainability

NFR12: Code must follow clean architecture principles.

NFR13: Unit test coverage should be at least 70%.

5. External Interface Requirements

5.1 User Interface

- Dashboard
- Product Management Page
- Sales Page
- Supplier Page
- Reports Page

5.2 Software Interfaces

- React frontend ↔ ASP.NET REST API
- ASP.NET ↔ MySQL (ADO.NET)
- GitHub Actions ↔ Azure App Service

6. System Architecture

3-Tier Architecture

1. Presentation Layer (React)
2. Business Logic Layer (ASP.NET)
3. Data Layer (MySQL)

7. CI/CD Requirements

The system shall:

- Automatically build on push to GitHub
- Run unit tests automatically
- Deploy to Azure automatically
- Send failure notifications

8. Testing Requirements

The system must include:

- Unit Testing (Backend)
- Integration Testing
- Selenium E2E testing
- JMeter Load Testing
- Code coverage reporting

9. Assumptions and Dependencies

- Internet access is available
- Azure services are accessible
- Users have modern web browsers

10. Future Enhancements

- Online customer ordering
- Mobile app
- Barcode scanner integration
- Payment gateway integration